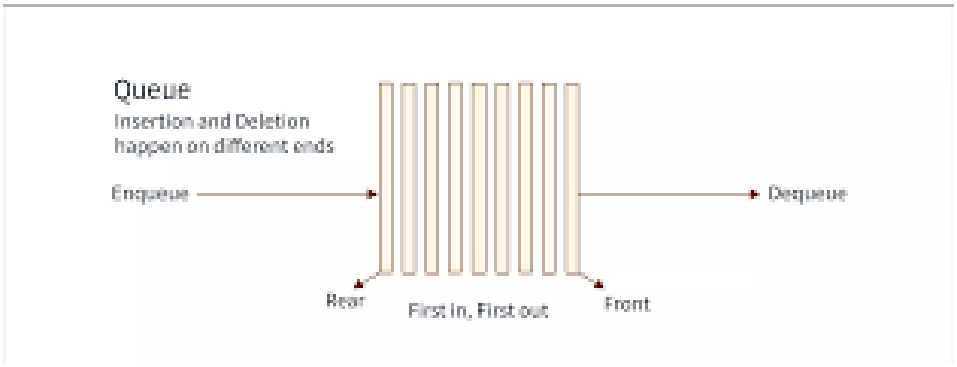
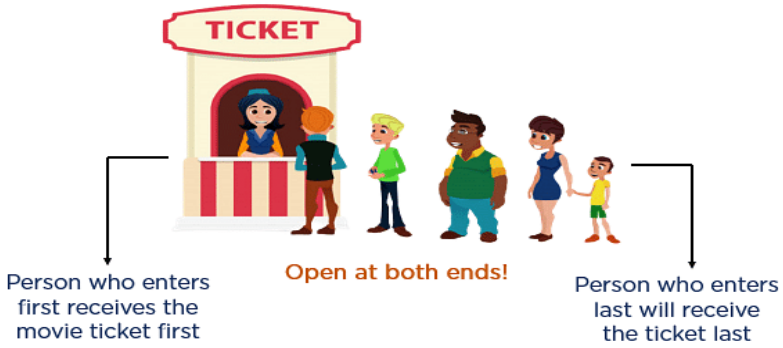
WEEK –10 FIRST-IN FIRST-OUT QUEUE DATA STRUCTURE

Queue is a linear data structure used to store the data. It works on the principle First-In First-Out. The queue has the two ends front and rear. The element is inserted from the **rear** end and removed from the **front** end.



Real world examples: vehicles in single-lane, one-way road, where vehicles enter first, exits first, queues at ticket counter and bus stops etc.





Types of Queue

1. Simple queue or linear queue
2. Circular queue
3. Priority queue
4. Double ended queue

Example :

1. **Media player**

The music player software allows user to add songs to playlist. All the songs in the playlist are played one after another i.e., sequentially after pressing play button. Playing of songs can be implemented using queue.

2. **Keyboard buffer queue**

Queue used in keyboard buffer makes the letter to appear on the screen in the order the user press them.

3. **Printer Queue**

When printer is shared, several people may send a print job to the printer. Queues used in printer allows to print in the order sent.

Implementing the queue and its operation in python using list

The advantage of using queue data structure to create queue is that -it is easy to implement and the list is limitless. The built-in functions of list append( ) and pop( ) is used. However, list implementation may become slower as the size of the list increases. The object-oriented feature of python that is class can also be used to implement queue.

Operations performed on queue

1. **Create queue**
   Queue is used to store data and can be implemented in python using list.

1. **Enqueue** – The Enqueue operation is used to insert data at the **rear** end of the queue.
2. **Dequeue** – The Dequeue operation is used to delete the data from the **front** end of the queue.
3. **Peek** – It returns the data pointed by front pointer in the queue but does not delete it.
4. **IsEmpty** – It returns whether a queue is empty or not. Returns Boolean value True if queue is empty, otherwise it returns False.
5. **IsFull -** It returns whether a queue is full or not. Returns Boolean value True if queue is full, otherwise it returns False.
6. **DeleteQueue** – Removes all the data stored in queue and free up the allocated memory space.

Enqueue Operation

The enqueue operation is used to insert elements into a queue. Elements are inserted from **rear** end. The built-in function “append" is used to enter elements in the list. Queue has limited size. The process of adding elements is stopped when queue is full.

The steps involved in the enqueue operation are –

1. Checks if the queue is full.
2. If the queue is full, display queue is full.
3. If the queue is not full, Add element to the queue.

Dequeue Operation

The dequeue operation is used to delete elements from a queue. Elements are deleted from **front** end. The built-in function “pop" is used to delete elements from the list.

Deleting of elements is stopped when queue is empty.

The steps involved in the push operation are –

1. Checks if the queue is empty.
2. If the queue is empty, display queue is empty.
3. If the queue is not empty, delete element from the queue.

Python Code:

```
def Enqueue():
    if len(que)==size: # check whether queue is full or not
        print("Queue is Full!!!!")
    else:
        element=input("Enter the element:")
        que.append(element)
        print(element,"is added to the Queue!")

def dequeue():
    if len(que) == 0 :   # Check whether queue is empty or not
        print("Queue is Empty!!!")
    else:
        ele=que.pop(0)
        print("element removed!!:",ele)

def display():
    print("Contents of queue : ", que)
```

```
# Main
que=[]
size=int(input("Enter the size of Queue:"))
while True:
    print("Select the Operation:1.Add 2.Delete 3. Display 4. Quit")
    choice=int(input())
    if choice == 1:
        Enqueue()
    elif choice == 2:
        dequeue()
    elif choice == 3:
        display()
    elif choice == 4:
        break
    else:
        print("Invalid Option!!!")
```

Output:

```
Enter the size of Queue:4
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
1
Enter the element:11
11 is added to the Queue!
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
1
Enter the element:22
22 is added to the Queue!
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
1
Enter the element:33
33 is added to the Queue!
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
1
Enter the element:44
44 is added to the Queue!
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
1
Queue is Full!!!!
```

```
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
3
Contents of queue :  ['11', '22', '33', '44']
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
2
element removed!!: 11
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
3
Contents of queue :  ['22', '33', '44']
Select the Operation:1.Add 2.Delete 3. Display 4. Quit
4
```

Time and Space Complexity for Operations on Queue built using List

	Time Complexity	SpaceComplexity
Create Queue	O(1)	O(1)
Enqueue	O(n)	O(1)
Dequeue	O(n)	O(1)
Peek	O(1)	O(1)
isEmpty	O(1)	O(1)

Priority Queue

Priority Queues is similar to queue, in which, each data/item is assigned a priority.

The priority of items determines the order in which elements are removed from the priority queue. Therefore, all the items are either arranged in an ascending or descending order.
So, a Priority Queue is an extension of the queue with the following properties.

1. In priority queue, data when inserted, is stored based on its priority.
2. An element with high priority is dequeued before an element with low priority.
3. If two elements have the same priority, they are served according to their order in the queue.
4. priority queue will not be following the basic principle of a queue, First in First Out (FIFO). Because a priority queue is an advanced queue used, where data is arranged and manipulated as per the given priority.

A priority queue can be of two types:

1. **Max Priority Queue**: Which arranges the data as per descending order of their priority.
2. **Min Priority Queue**: Which arranges the data as per ascending order of their priority.

Applications of the Priority queue

- Job Scheduling algorithms
- CPU and Disk Scheduling
- Managing resources that are shared among different processes
- Solving Dijkstra's Algorithm, Prim's algorithm

Implementing Priority Queue

Priority queue can be implemented using python **list** and object oriented concept.

Algorithm steps:

1. The **Node** class describes the element of the priority queue.
2. Define **Insert** method in **PriorityQueue Class** to add a new data element in the priority queue.
    - If the priority queue is empty, insert the element to it.
    - If the priority queue is not empty, traverse the queue, comparing the priorities of the existing nodes with the new node, and add the new node before the node with priority greater than the new node.
    - If the new node has the highest priority, then add the new node at the end.
3. Define **Delete** method to remove the element with least priority.
4. Define **Display** method to print all the priority queue elements.

Python Code:

```
class Node:
    def __init__(self,info,priority):
        self.info = info
        self.priority = priority

class PriorityQueue:
    def __init__(self):
        self.queue = []

    def Insert(self,n):
        if len(self.queue) == 0:
            self.queue.append(n)
        else:
            for x in range(0, (len(self.queue))):
                if n.priority >= self.queue[x].priority:
                    if x == len(self.queue)-1:
                        self.queue.insert(x + 1, n)
                    else:
                        continue
                else:
                    self.queue.insert(x,n)
                    return True
```

```
    def Delete(self):
        return self.queue.pop(0)

    def Display(self):
        for x in self.queue:
            print(x.info, "-",x.priority)
```

```
# Main
pqueue = PriorityQueue()
n1 = Node("C",3)
n2 = Node("B",2)
n3 = Node("A",1)
n4 = Node("Z",26)
n5 = Node("L",12)
n6 = Node("M",18)

pqueue.Insert(n1)
pqueue.Insert(n2)
pqueue.Insert(n3)
pqueue.Insert(n4)
pqueue.Insert(n5)
pqueue.Insert(n6)
print("After Inserting .... ")
pqueue.Display()

print("After Deleting .... ")
pqueue.Delete()
pqueue.Display()
```

Output:

```
After Inserting ....
A - 1
B - 2
C - 3
L - 12
M - 18
Z - 26
After Deleting ....
B - 2
C - 3
L - 12
M - 18
Z - 26
```

Time Complexity of priority queue

The list is so created so that the highest priority element is always at the head of the list. The list is arranged in descending order of elements based on their priority. This

allow us to **remove** the highest priority element in **O(1)** time. To insert an element, we must traverse the list and find the proper position to insert the node so that the overall order of the priority queue is maintained. This makes the **Insert()** operation takes **O(n)** time. The Delete( ) operations is performed in constant time.

Differences between Priority Queue and Queue

	Priority Queue	Queue
1	In a priority queue, the values are removed **on the basis of priority**	In a queue, the **first-in-first-out rule** is implemented
2	When elements are popped out of a priority queue the result obtained is either sorted in Increasing order or in Decreasing Order	When elements are popped from a simple queue, a FIFO order of data is obtained in the result.

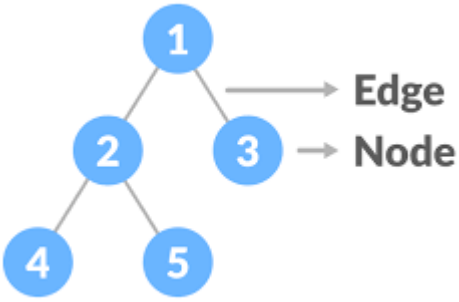
WEEK –11 THE TREE DATA STRUCTURE

Tree is a non-linear data structure. Tree data structure is a collection of data (Node) which organizes data in hierarchical structure recursively.

In tree data structure, every individual element is called as **Node**. Node stores the data of that particular element and link to next element in hierarchical structure. Tree consist of single root and one or more sub-trees attached to it.

Tree is an **acyclic connected** graph.

Forest is **disconnected acyclic** (do not contain self-loops) graph.



Example:

1. File Explorer / Folder Structure

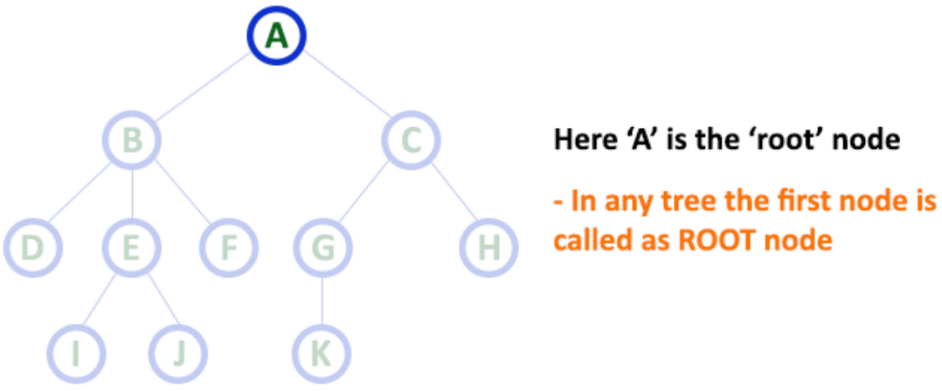
File explorer in windows helps to get the files quickly and easily. It provides a graphical interface for the user to navigate and access the files stored in the computer. Files and folders are organized in a hierarchical tree structure.

Tree Terminology

In a tree data structure, the following terminology are used...

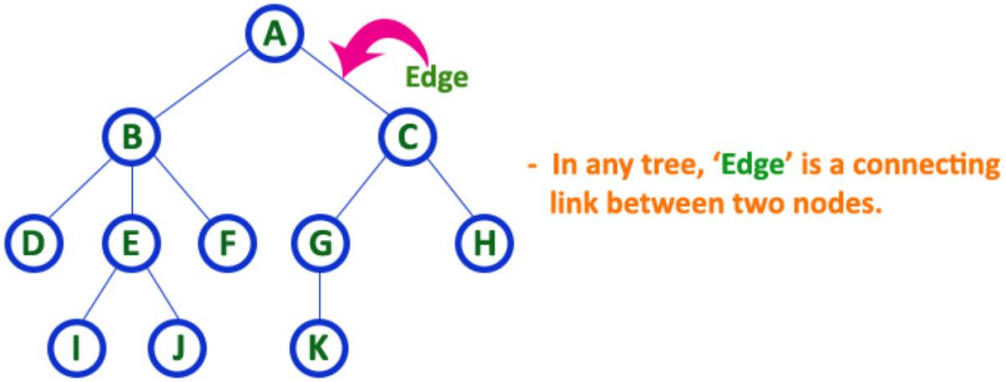
1. Root

In a tree data structure, the first node is called as **Root Node**. Every tree must have a root node. Root node is the origin of the tree data structure. In any tree, there must be only one root node. Multiple root nodes are not allowed in a tree.



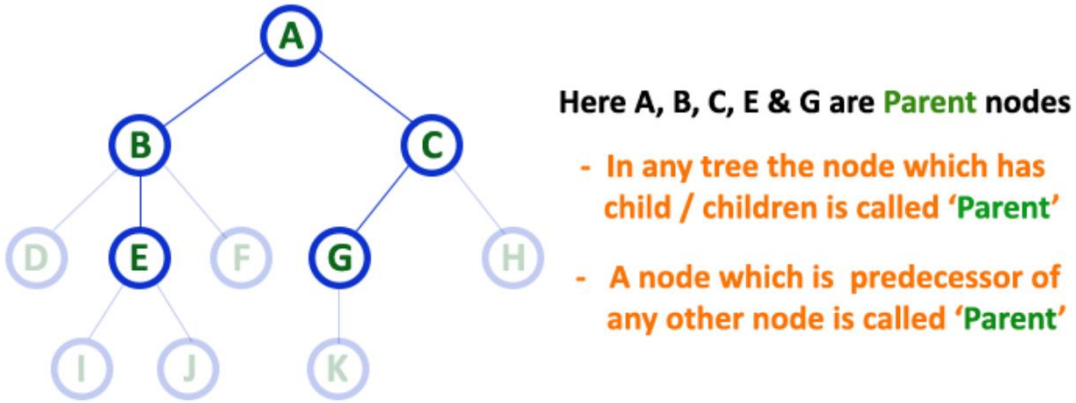
2. Edge

In a tree data structure, the connecting link between any two nodes is called as **EDGE**. In a tree with '**N**' number of nodes there will be a maximum of '**N-1**' number of edges.



3. Parent

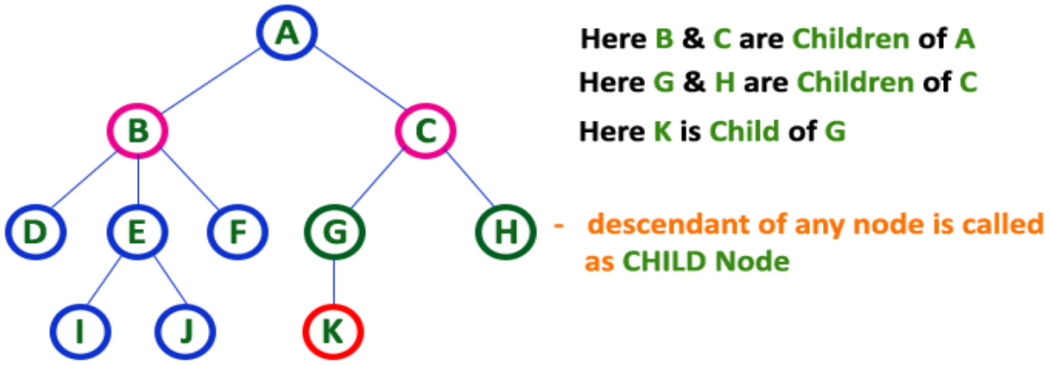
In a tree data structure, the node which is a predecessor of any node is called as **PARENT NODE**. Parent node can also be defined as "**The node which has child / children**".



4. Child

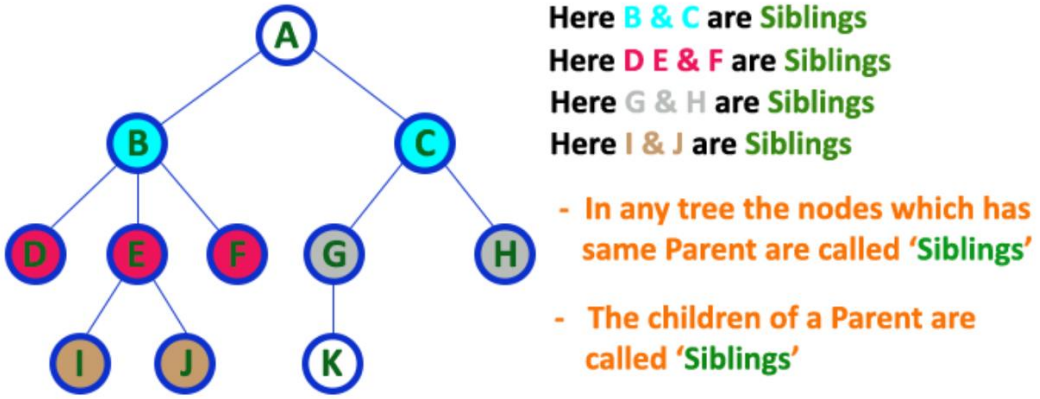
In a tree data structure, the node which is descendant of any node is called as **CHILD Node**. The node which has a link from its parent node is called as **child node**. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes

except root are child nodes.



5. Siblings

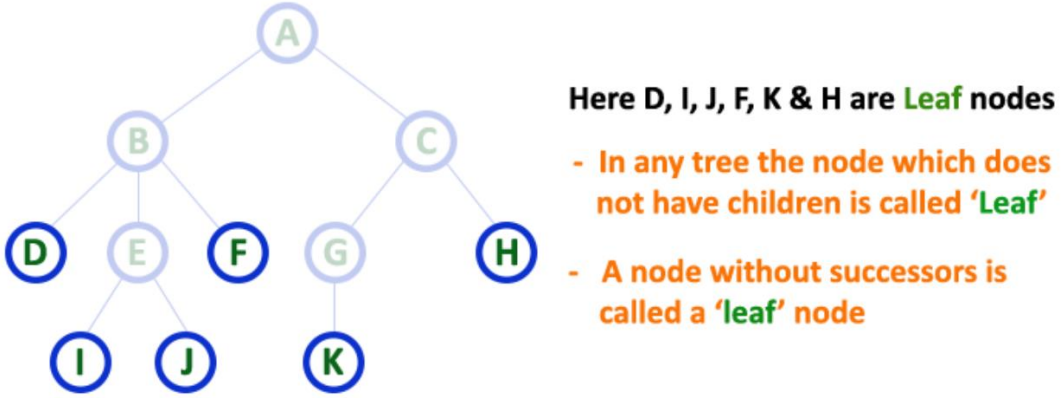
In a tree data structure, nodes of same parent are called **Siblings**.



6. Leaf

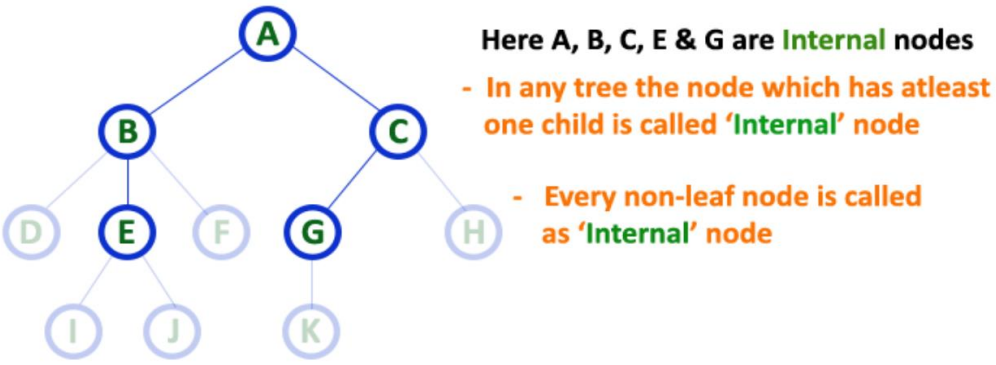
In a tree data structure, a leaf is a node with no child.

The leaf nodes are also called as **External Nodes or Terminal Node**. External node is also a node with no child.



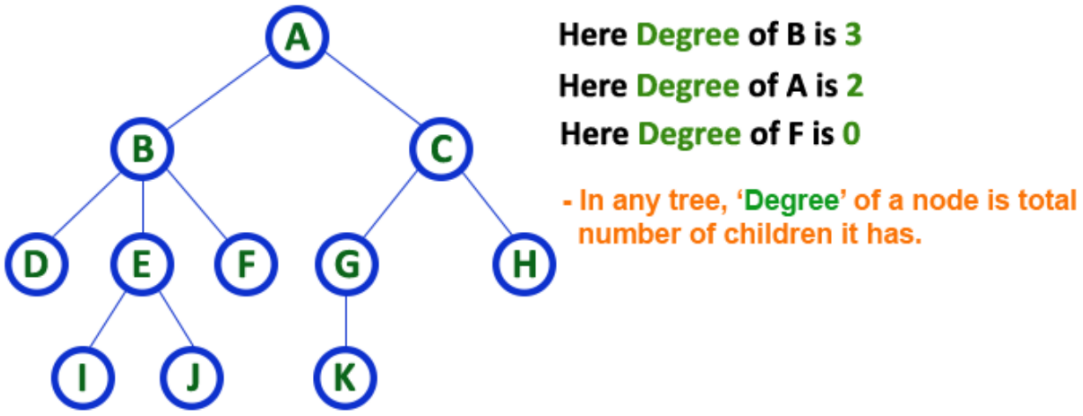
7. Internal Nodes

In a tree data structure, the node with at least one child is called as **INTERNAL Node**. Nodes other than leaf nodes are called as **Internal Nodes**. **The root node also called Internal Node** if the tree has more than one node. Internal nodes are also called as 'N**on-Terminal**' nodes.



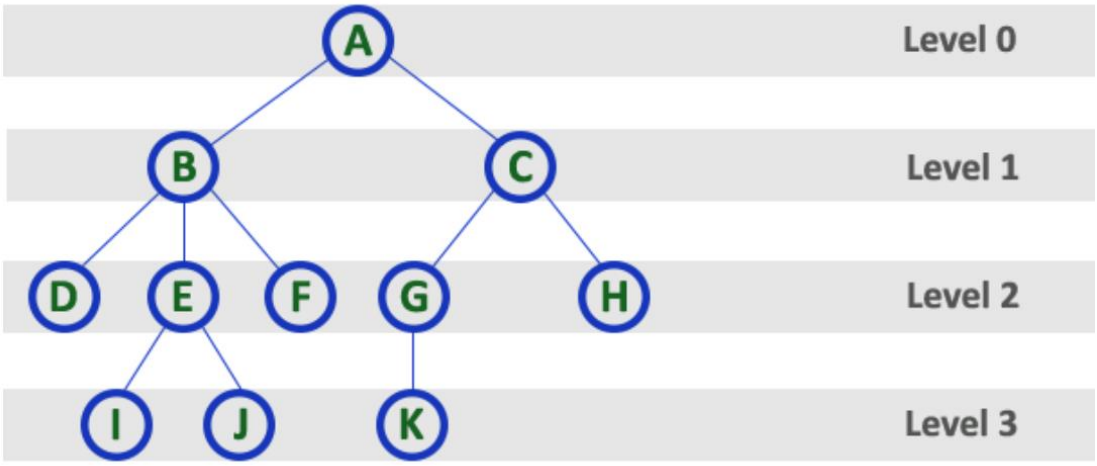
8. Degree

In a tree data structure, the total number of children of a node is called as **DEGREE** of that Node. The highest degree of a node among all the nodes in a tree is called as '**Degree of Tree**'.



9. Level

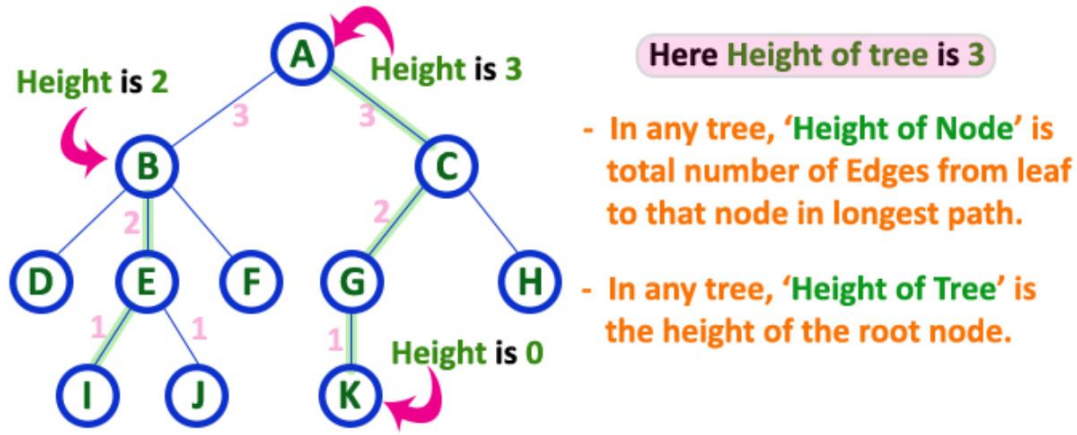
In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on. In a tree each step from top to bottom is called a Level and the Level count starts with '0' and incremented by one at each level (Step).



10. Height

In a tree data structure, the total number of edges from leaf node to a particular node in the longest path is called as **HEIGHT** of that Node.

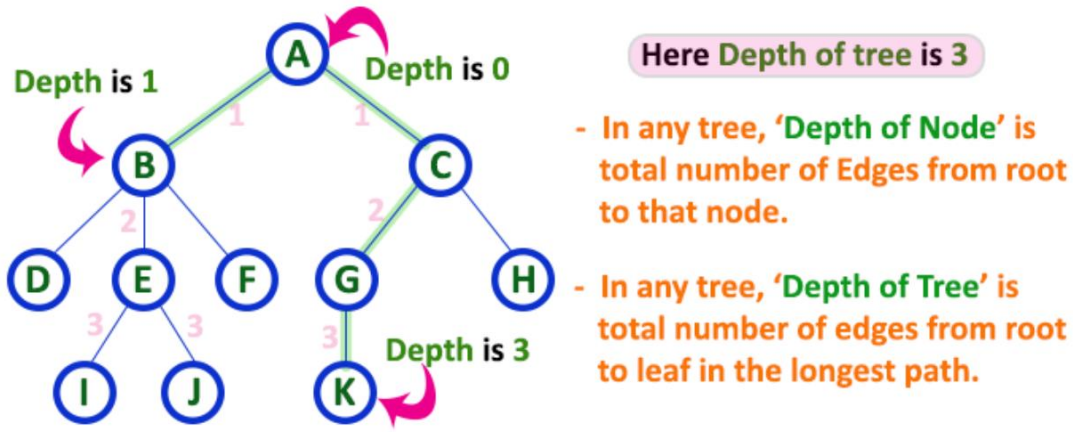
Height of the root node is said to be **height of the tree**. In a tree, **height of all leaf nodes is '0'.**



11. Depth

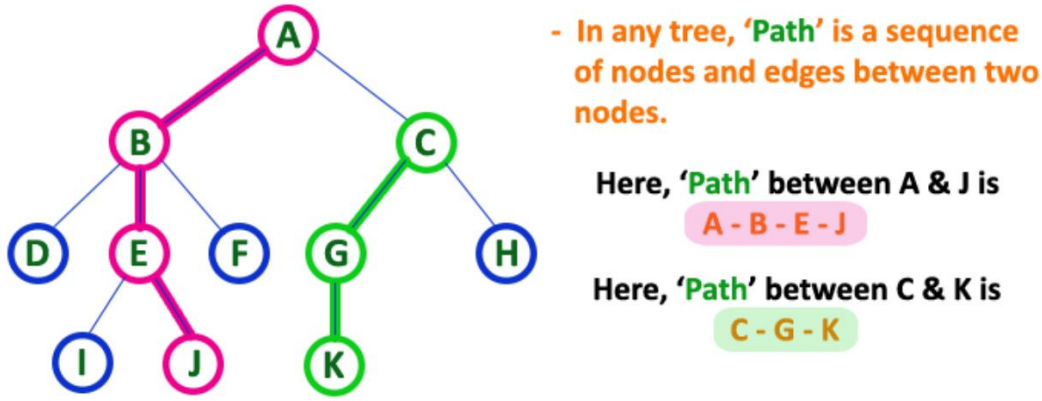
In a tree data structure, the total number of edges from root node to a particular node is called as **DEPTH** of that Node.

In a tree, the total number of edges from root node to a leaf node in the longest path is said to be **Depth of the tree**.



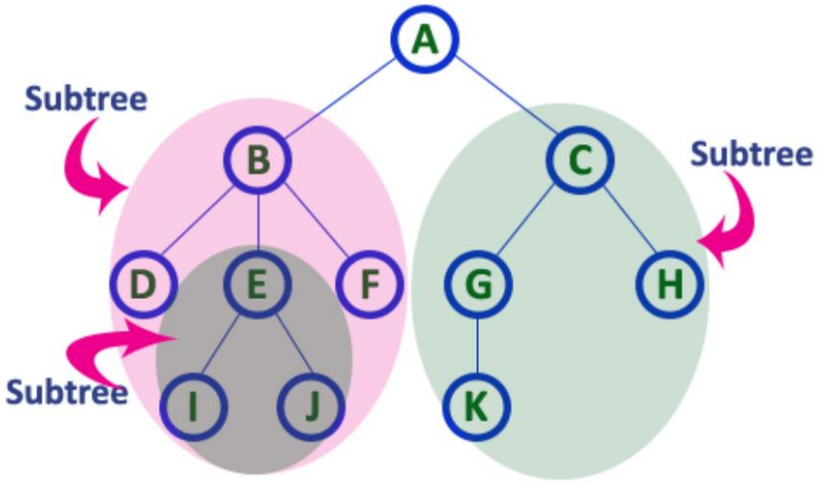
12. Path

In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as **PATH** between that two Nodes.



13. Sub Tree

In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.



Binary Tree

In a tree, every node can have any number of children. A **binary tree** is a special type of tree data structure in which every node can have a **maximum of 2 children**. One is known as a left child and the other is known as right child.

A tree in which every node can have a maximum of two children is called Binary Tree.

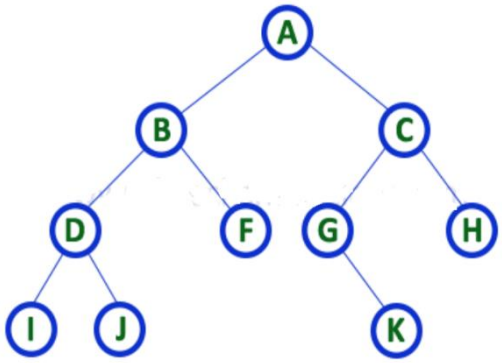
In a binary tree, every node can have either 0 children or 1 child or 2 children but not more than 2 children.

Binary Tree Representation

A binary tree data structure is represented using two methods. Those methods are as follows...

1. **Array Representation**
2. **Linked List Representation**

Consider the binary tree...



Array Representation of a Binary Tree

One-dimensional array (1-D Array) is used to represent a binary tree.
Consider the above example of a binary tree and it is represented as follows...

A	B	C	D	F	G	H	I	J	-	-	-	K	-	-	-	-	-	-	-	-

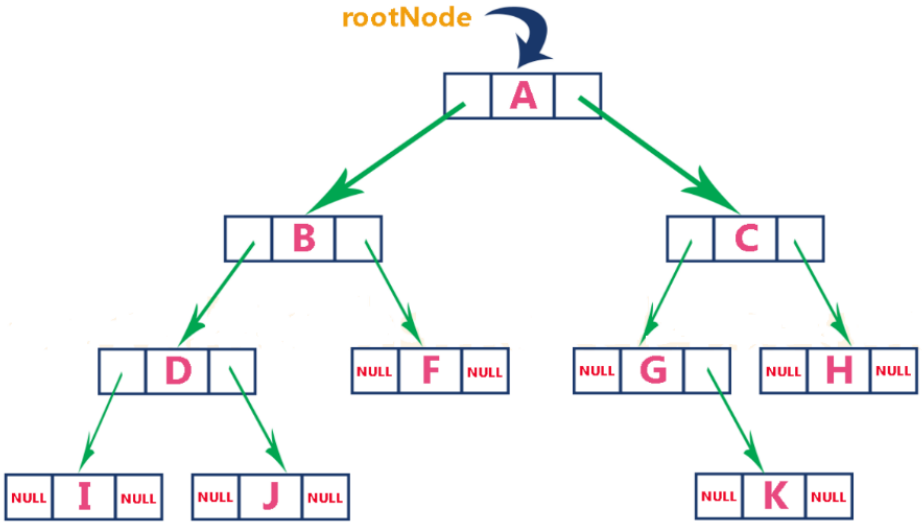
To represent a binary tree of depth **'n'** using array representation, one dimensional array with a maximum size of **2n + 1** is needed**.**

Linked List Representation of a Binary Tree

Doubly linked list is used to represent a binary tree. In a doubly linked list, every node consists of three fields. First field for storing left child address, second for storing actual data and third for storing right child address.
In linked list representation, a node has the following structure...



The above example of the binary tree represented using Linked list representation is shown as follows...



Types of Binary Tree

1. Strictly Binary Tree
2. Balanced Binary Tree
3. Complete Binary Tree

There are other data structures that are derived from binary tree, such as binary search tree, hash tree, AVL Tree etc.

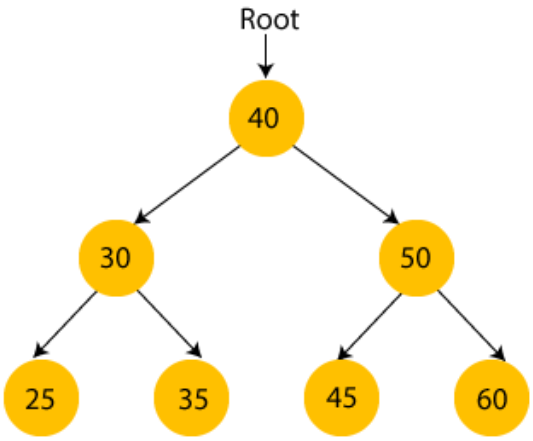
Binary Search Tree

A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

Properties of Binary Tree

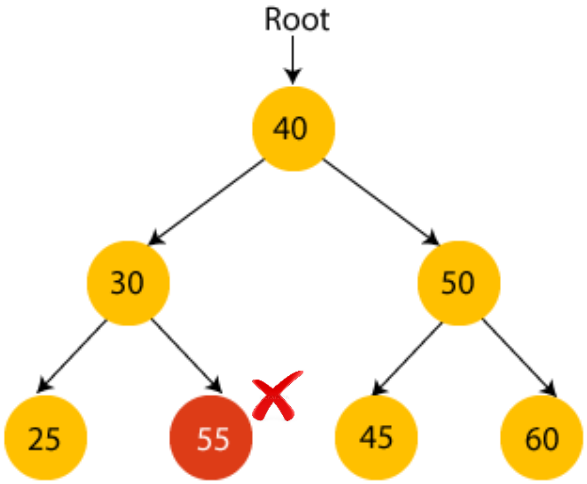
1. Every binary search tree is a binary tree.
2. Every left child will hold lesser value than root.
3. Every right child will hold greater value than root.
4. Ideally binary search tree will not hold same value twice.

Example 1:



In figure, the root node is 40, and all the nodes of the left subtree are smaller than the root node, and all the nodes of the right subtree are greater than the root node.

Example 2: Suppose if we change the value of node 35 to 55 in the above tree, check whether the tree will be binary search tree or not.



In the above tree, the value of root node is 40, which is greater than its left child 30 but smaller than right child of 30, i.e., 55. So, the above tree does not satisfy the property of Binary search tree. Therefore, the above tree is not a binary search tree.

Advantages of Binary Tree

- Searching an element in the Binary search tree is easy
- As compared to array and linked lists, insertion and deletion operations are faster in BST.

Example of creating a binary search tree

Suppose the data elements are **- 45, 15, 79, 90, 10, 55, 12, 20, 50**

- First, insert **45** into the tree as the root of the tree.
- Then, read the next element; if it is smaller than the root node, insert it as the root of the left subtree, and move to the next element.

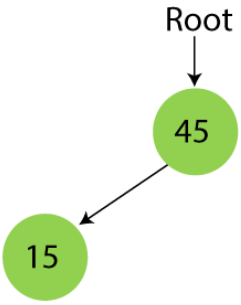
- Otherwise, if the element is larger than the root node, then insert it as the root of the right subtree.

Step 1 - Insert 45.



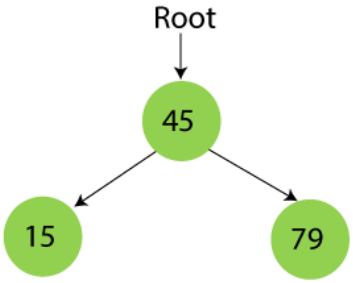
Step 2 - Insert 15.

As 15 is smaller than 45, so insert it as the root node of the left subtree.



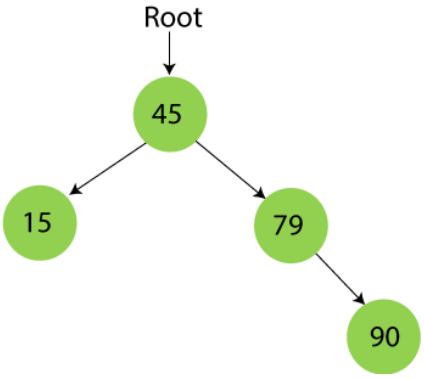
Step 3 - Insert 79.

As 79 is greater than 45, so insert it as the root node of the right subtree.



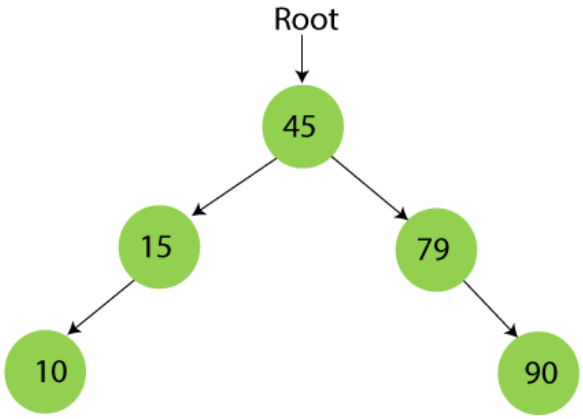
Step 4 - Insert 90.

90 is greater than 45 and 79, so it will be inserted as the right subtree of 79.



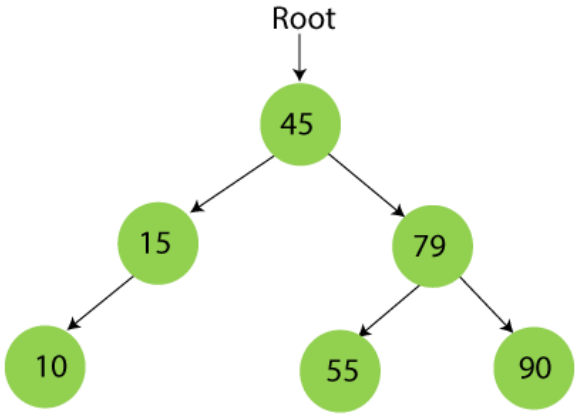
Step 5 - Insert 10.

10 is smaller than 45 and 15, so it will be inserted as a left subtree of 15.



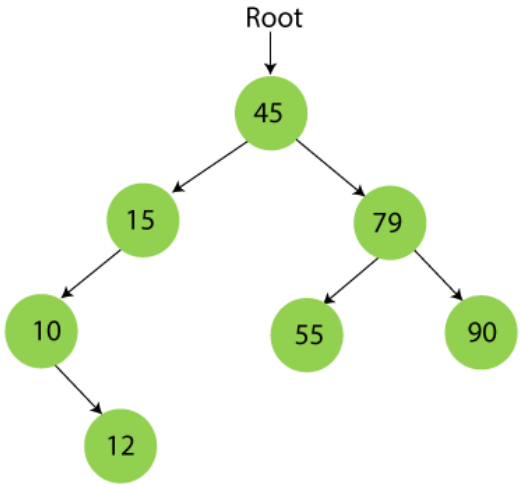
Step 6 - Insert 55.

55 is larger than 45 and smaller than 79, so it will be inserted as the left subtree of 79.



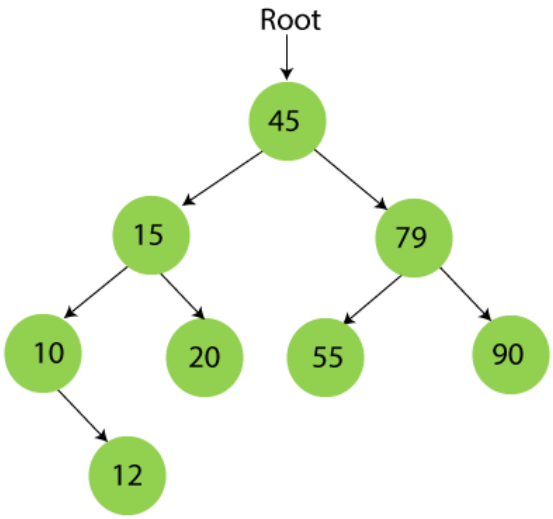
Step 7 - Insert 12.

12 is smaller than 45 and 15 but greater than 10, so it will be inserted as the right subtree of 10.



Step 8 - Insert 20.

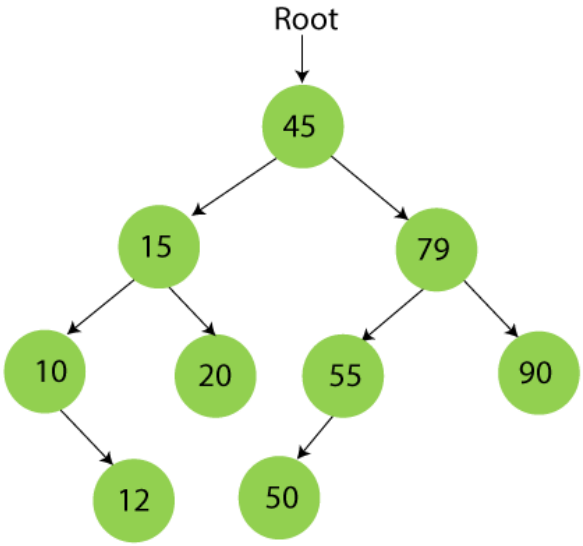
20 is smaller than 45 but greater than 15, so it will be inserted as the right subtree of 15.



Step 9 - Insert 50.

50 is greater than 45 but smaller than 79 and 55. So, it will be inserted as a left subtree of 55.

Now, the creation of binary search tree is completed.



Implementation of Tree Operations

Creation

1. Create BST class to initialize binary search tree.
2. Initialize BST class 3 variables- data, reference to left child and reference to right child node

```
class BST:
    def __init__(self, data):
        self.left = None
        self.right = None
        self.data = data
```

Time complexity for creation of binary search tree is O(1) because it takes constant time for the initialization of data variables. **Space complexity is O(1)** since no additional memory is used.

Insertion of Node

The process of insertion of node in a binary tree is ordered. This means that while inserting a node in a binary search tree, find the right place where the node must be inserted according to the value it holds. The following are the steps to insert a node –

- If the binary tree is empty, insert the new node and set it as the root of the binary search tree.
- If the value of the new node is lesser than the value of the parent and root, move to the left subtree and continue the process of comparison until the correct position is found.
- If the value of the new node is greater than the value of the parent and root, move to the right subtree and continue the process of comparison until the correct position is found.

Python Code:

```
# Insert method to create nodes
   def insert(self, data):
      if self.data == None:
          return

      if data < self.data:
            if self.left == None:
               self.left = BST(data)
            else:
               self.left.insert(data)
      else:
            if self.right == None:
               self.right = BST(data)
            else:
               self.right.insert(data)
```

Time and Space complexity for insertion of node

The recursive function searches for the correct position by dividing the subtrees at each level and moving on to the next level depends on the value of the node. The number of function calls takes the form of logN. Hence the **time complexity** for insertion is O(log N).

The space complexity is O(log N) because every time the function is called, it allocates space in the stack in memory for next function call.

Searching for a node

In a binary tree, the left child of a node has a value lesser than that of root and parent. The right child has a greater value. Thus, while searching, compare the value to be searched with that of the node that is visited. If the value is less than that of node, traverse to the left subtree. Otherwise, traverse to the right subtree. This optimizes the time utilization as the tree is divided at each level during the search.

Python Code:

```
def search(self, key):
    if self.data == key:
        print("\nSearching --Node is present in the tree")
        return

    if key < self.data:
        if self.left != None:
            self.left.search(key)
        else:
            print("\nSearching --Node is not present in  the tree")

    else:
        if self.right != None:
            self.right.search(key)
        else:
            print("\nSearching --Node is not present in  the tree")
```

Time and Space complexity for searching a node

The recursive function searches for the correct node by dividing the subtrees at each level and moving on to the next level depends on the value of the node. The number of function calls takes the form of logN. Hence the **time complexity** for search is O(log N).

The **space complexity** is O(log N) because every time the function is called, it allocates space in the stack in memory for next function call.

Tree Traversal

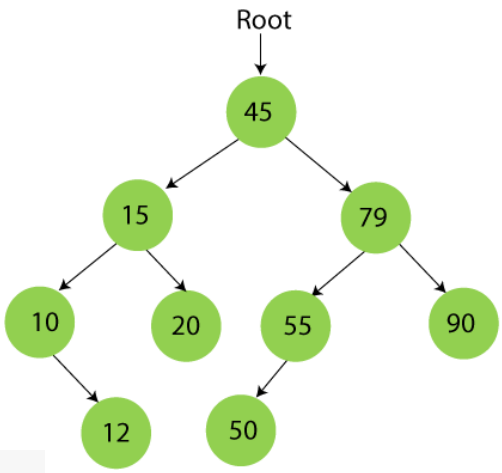
Traversal is the process of visiting every node in the tree, exactly once, to print values, search nodes etc.

There are 3 ways to traverse a tree −

- In-order Traversal
- Pre-order Traversal
- Post-order Traversal

In-order Traversal in Binary Search Tree

1. First, visit all the nodes in the left subtree
2. Then visit the root node
3. Visit all the nodes in the right subtree



In-order traversal ----

10 12 15 20 45 50 55 79 90

Implementation using python code

```
def inorder(self):
    if self.left != None:
        self.left.inorder()

    print(self.data, end="  ")

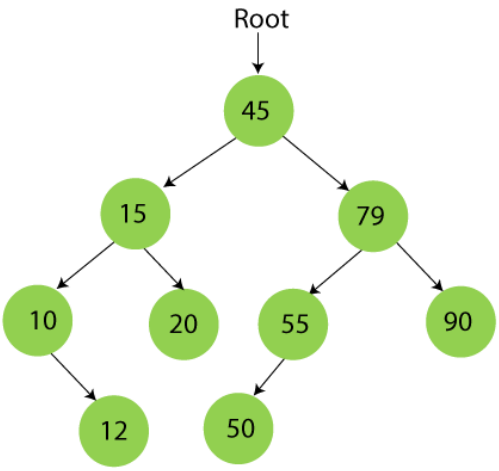
    if self.right != None:
        self.right.inorder()
```

Time and space complexity for in-order traversal

The time complexity for in-order traversal is O(n) because the function recursively visits all nodes of the tree.

The space complexity for in-order traversal is O(n) because when a function is called recursively stack is used to store function calls.

Pre-order Traversal in Binary Search Tree



1. Visit the root node
2. Visit all the nodes in the left subtree
3. Visit all the nodes in the right subtree

Pre-order traversal ----

45 15 10 12 20 79 55 50 90

Implementation using python code

```
def preorder(self):
    print(self.data, end="  ")

    if self.left != None:
        self.left.preorder()

    if self.right != None:
        self.right.preorder()
```

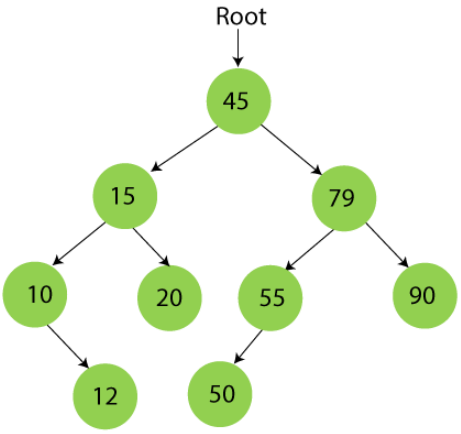
Time and space complexity for pre-order traversal

The time complexity for in-order traversal is O(n) because the function recursively visits all nodes of the tree.

The space complexity for in-order traversal is O(n) because when a function is called recursively stack is used to store function calls.

Post-order Traversal in Binary Search Tree

1. Visit all the nodes in the left subtree
2. Visit all the nodes in the right subtree
3. Visit the root node



Post-order traversal ----

12 10 20 15 50 55 90 79 45

Implementation using python code

```
def postorder(self):
    if self.left != None:
        self.left.postorder()

    if self.right != None:
        self.right.postorder()

    print(self.data, end="  ")
```

Time and space complexity for post-order traversal

The time complexity for in-order traversal is O(n) because the function recursively visits all nodes of the tree.

The space complexity for in-order traversal is O(n) because when a function is called recursively stack is used to store function calls.

	Time complexity	Space complexity
Create BST	O(1)	O(1)
Inserting a node	O(log N)	O(log N)
Search a node	O(log N)	O(log N)
Delete a node	O(log N)	O(log N)
Traverse BST	O(N)	O(N)

~~~~~~~~~~~~~~~~~~~~**********~~~~~~~~~~~~~~~~
~~~~~~~~~~~~~~~~~~~~

Week –12 Depth first traversal

Depth-First Search is an algorithm for traversing or searching tree or graph data structure. It is used to traverse graph nodes in a top to down orderly fashion. Using DFS, one can visit all the nodes in a graph that are reachable from the starting node.

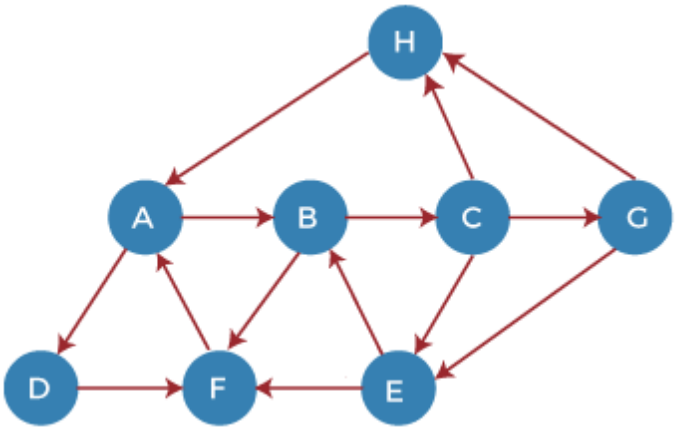
To implement the DFS algorithm – visited list and stack is used.

- **Visited list** - is used to check whether or not all the nodes of a graph are visited or not.
- **Stack** - is used to push adjacent and unvisited vertices of a graph.

The DFS algorithm works as follows:

1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Repeat steps 2 and 3 until the stack is empty.

Example: The example below shows how DFS algorithm works.



Adjacency Lists

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

Let the starting Node of the graph be H.

Step 1 - First, push H onto the stack.

Stack: H

Step 2 - POP the top element from the stack, i.e., H, and print it. Now, PUSH all the neighbours of H onto the stack.

print: H

Stack: A

Step 3 - POP the top element from the stack, i.e., A, and print it. Now, PUSH all the neighbours of A onto the stack.

print: A

Stack: B, D

Step 4 - POP the top element from the stack, i.e., D, and print it. Now, PUSH all the neighbours of D onto the stack.

Print: D

Stack: B, F

Step 5 - POP the top element from the stack, i.e., F, and print it. Now, PUSH all the neighbours of F onto the stack.

Print: F

Stack: B

Step 6 - POP the top element from the stack, i.e., B, and print it. Now, PUSH all the neighbours of B onto the stack.

Print: B

Stack: C

Step 7 - POP the top element from the stack, i.e., C, and print it. Now, PUSH all the neighbours of C onto the stack.

Print: C

Stack: E, G

Step 8 - POP the top element from the stack, i.e., G and PUSH all the neighbours of G onto the stack that are in ready state.

Print: G

Stack: E

Step 9 - POP the top element from the stack, i.e., E and PUSH all the neighbours of E onto the stack.

Print: E

Stack:

Now, all the graph nodes have been traversed, and the stack is empty.

Application of DFS

- Used to implement the topological sorting.
- To find the paths between two vertices.
- Used to detect cycles in the graph.
- To find connected components of a graph
- For detecting cycles in a graph

Python Code:

```
def dfs(graph, source):
    stack = []
    visited = []
    stack.append(source)
    visited.append(source)

    while len(stack) != 0:  # Stack not equal to empty
        vertex = stack.pop()
        print(vertex, end="  ")
        for u in graph[vertex]:
            if u not in visited:
                stack.append(u)
                visited.append(u)

# Main
graph = { 'A':['B', 'D'], 'B':['C', 'F'], 'C':['E', 'G', 'H'],
        'D':['F'], 'E':['B', 'F'], 'F':['A'], 'G':['F'], 'H':['A']}

print("Given Graph" , graph)

print("\n DFS Traversal \n")
dfs(graph, "H")
```

Output:

```
Given Graph {'A': ['B', 'D'], 'B': ['C', 'F'], 'C': ['E', 'G', 'H'], 'D': ['F'], 'E': ['B', 'F'],
'F': ['A'], 'G': ['F'], 'H': ['A']}

 DFS Traversal

H  A  D  F  B  C  G  E
```

Time and Space Complexity of Depth First Search

The **time complexity** of the DFS algorithm is represented in the form of O (V +E), where V is the number of nodes and E is the number of edges.

The **space complexity** of the algorithm is O(V).

Breadth First Search

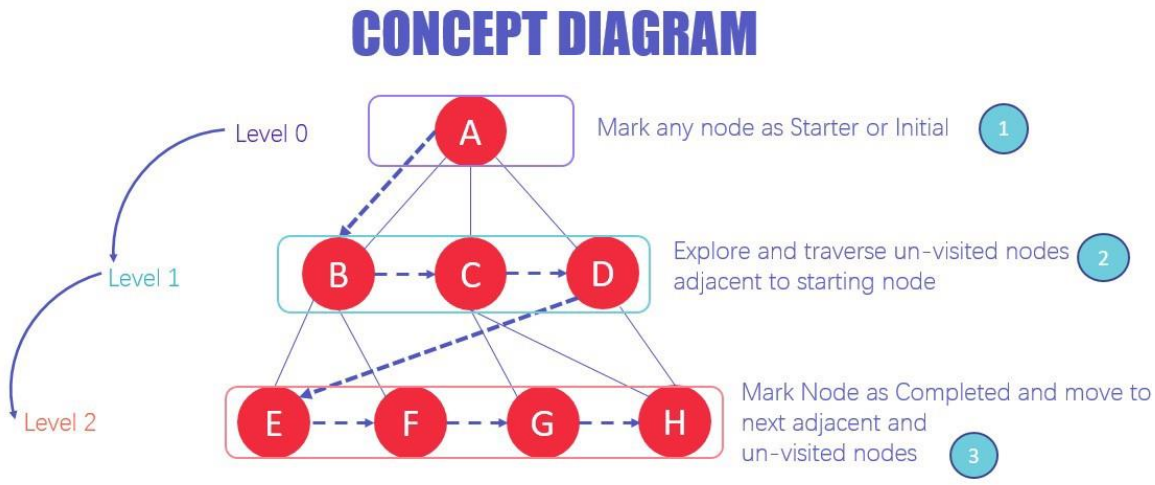
Breadth first search algorithm is graph traversal algorithm. It is used to traverse graph in an orderly fashion. Using BFS, one can visit all the nodes in a graph that are reachable from the starting node.

To implement the BFS algorithm – visited list and queue is used.

- **Visited list** - is used to check whether or not all the nodes of a graph are visited or not.
- **A queue** - is used to enqueue adjacent and unvisited vertices of a graph.

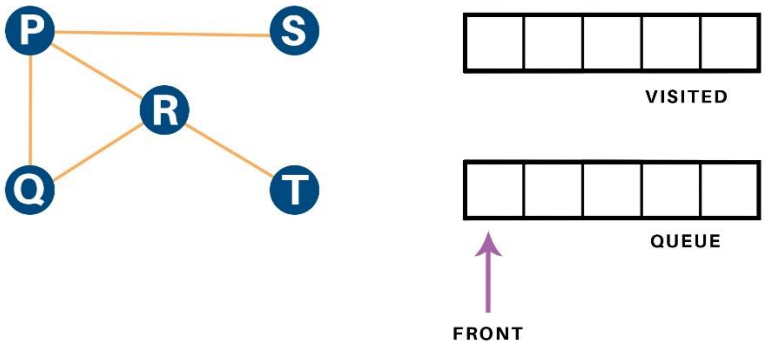
The algorithm works as follows:

1. Start by putting any one of the graph's vertices at the rear end of a queue.
2. Take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.

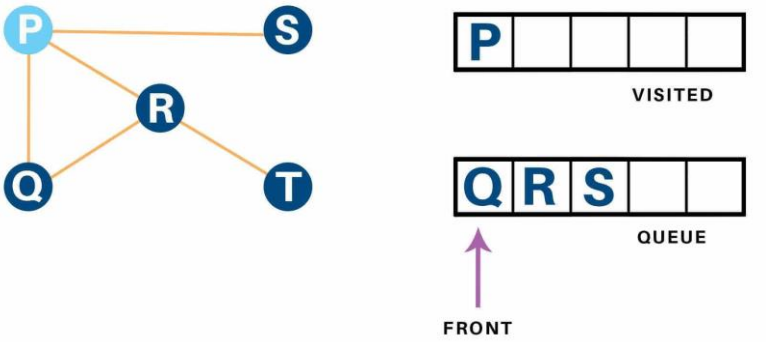


Example

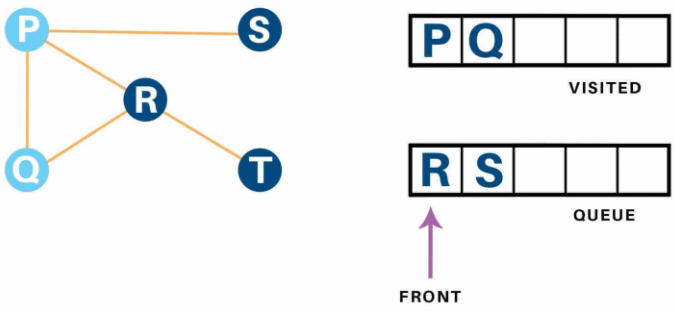
The example below shows how BFS algorithm works. Consider an undirected graph with 5 vertices.



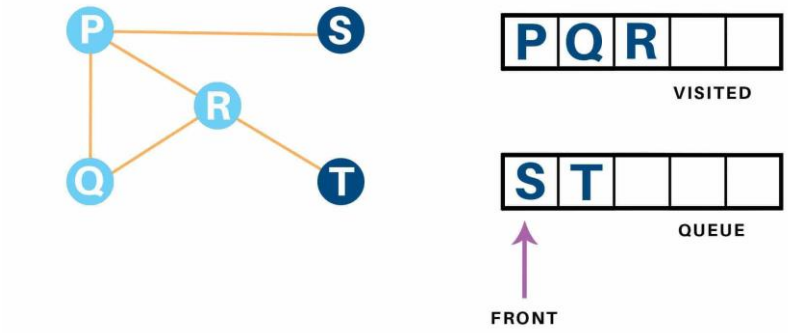
Let the starting vertex be P. The BFS algorithm starts by putting P in Visited list and puts all its adjacent vertices in queue.



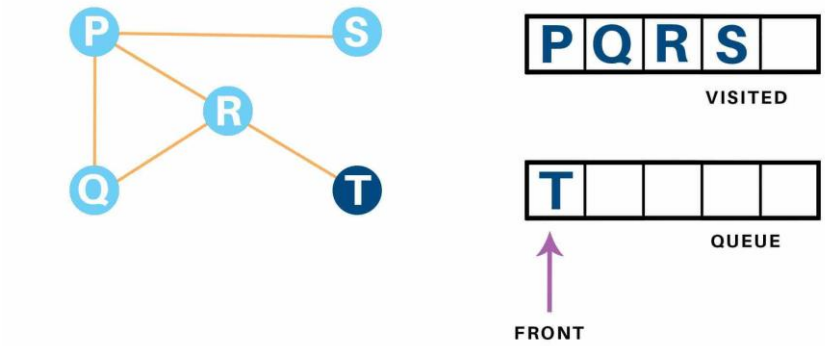
Next, visit from the front of the queue i.e. Q and visit its adjacent nodes. The adjacent nodes of Q are P and R. Since P has already been visited, Next visit R.



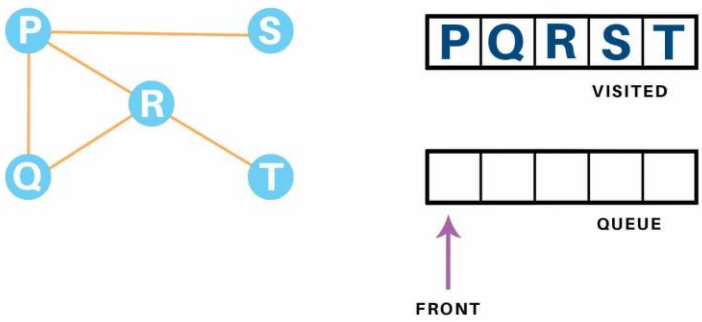
The adjacent Vertex R are P,Q,T. It has an unvisited adjacent vertex T, thus add T to the rear of the queue.



visit S, which is at the front of the queue. The adjacent vertex of S is P, which is already visited.



Now, only T is remaining in the queue. The adjacent vertex of T is R, which is already visited. Now, the visited list is full and queue is empty.



Since the queue is empty, the Traversal of the graph is completed.

Python Code

```
from queue import Queue

def bfs(graph, source):
    que = Queue()
    visited = []
    que.put(source)
    visited.append(source)

    while not que.empty():
        vertex = que.get()
        print(vertex, end="  ")
        for u in graph[vertex]:
            if u not in visited:
                que.put(u)
                visited.append(u)
```

```
# Main
graph = { 'A':['B', 'D'], 'B':['C', 'F'], 'C':['E', 'G', 'H'],
         'D':['F'], 'E':['B', 'F'], 'F':['A'], 'G':['F'], 'H':['A']}

print("Given Graph" , graph)

print("\n BFS Traversal \n")
bfs(graph, "H")
```

Output:

```
Given Graph {'A': ['B', 'D'], 'B': ['C', 'F'], 'C': ['E', 'G', 'H'], 'D': ['F'], 'E': ['B', 'F'],
'F': ['A'], 'G': ['F'], 'H': ['A']}

 BFS Traversal

H  A  B  D  C  F  E  G
```

Time Complexity and Space Complexity of BFS

The **time complexity** of the Breadth first Search algorithm is in the form of O(V+E), where V is the number of nodes and E is the number of edges.

The **space complexity** of the BFS algorithm is O(V).

Applications

Breadth-first Search Algorithm has a wide range of applications in the real-world. Some of them are:

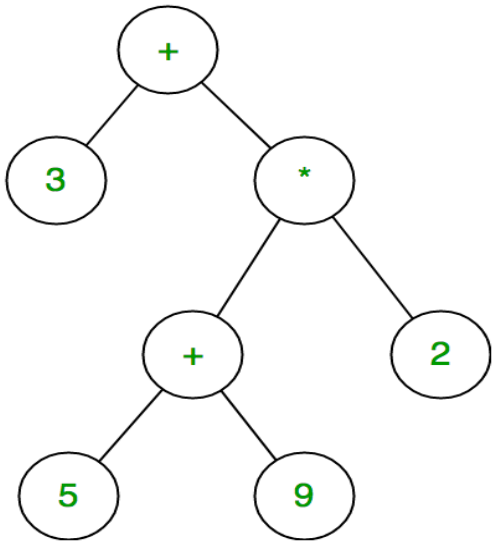
In GPS navigation, it helps in finding the shortest path available from one point to another.

1. In pathfinding algorithms
2. Cycle detection in an undirected graph
3. In minimum spanning tree
4. To build index by search index

Tree Application

An arithmetic expression is a sequence of tokens. A token may be either an operand or an operator. A binary arithmetic operation using arithmetic operators addition, subtraction, multiplication, division may be in the form of operand-operator- operand. For example 2+3, where 2 and 3 are operands, + is an addition operator.
An expression tree is a binary tree with the following properties-

- Each leaf is an operand
- The root and internal nodes are operators.
- Subtree are sub-expressions, with root being an operator.



Example:
Graph represents an expression
3 + ((5 + 9) * 2)
After evaluation, the output is 31

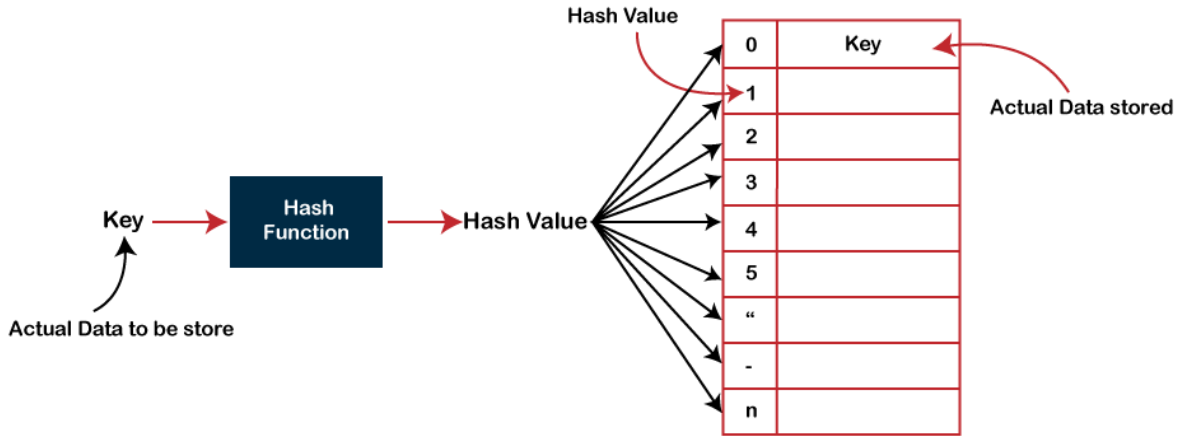
Week –13 Hashing

Hashing is a data structure. It is a technique of mapping a large chunk of data into small tables using a hashing function.
Hashing is one of the searching techniques that uses a constant time. The time complexity in hashing is O(1).

Hashing maps keys and values into the hash table by using a hash function. If the key is given, then the algorithm computes the index (Hash value) at which the value would be stored. It can be written as:

Index = hash(key)

The efficiency of mapping depends on the efficiency of the hash function used.



Terminology used in hashing

- **Key –** key is the data input by the user.
- **Hash value –** Hash value is the value returned by the hashing function.
- **Hash function –** Hash function is a function that is used to map data of arbitrary size(key) to hash value.

  Properties of an efficient hash function are:

  1. Hash function should be very fast in calculating and delivering the result.
  2. The result generated by the hash function should be unique and accurate

- **Hash table –** Hash table is a data structure that stores key-value pair. In python, hash table is implemented using dictionary data type. Hash values are integers.
- **Collision** – A collision occurs when two different keys to a hash function produce the same hash value.

Perfect hash function

A perfect hash function **h** for a set **S** is a hash function that maps distinct elements in **S** to a set of **m** integers, with no collision.

A minimal perfect hash function does so by using table that has as many slots as there are key values to be hashed.

There are three ways of calculating the hash function:

- Division method
- Folding method
- Mid square method

Hash Collision

Hash collision is a situation where 2 or more keys result in a same hash value. The situation where a new key maps to an already occupied slot in a hash table.

To resolve these collisions, there are some techniques known as collision techniques.

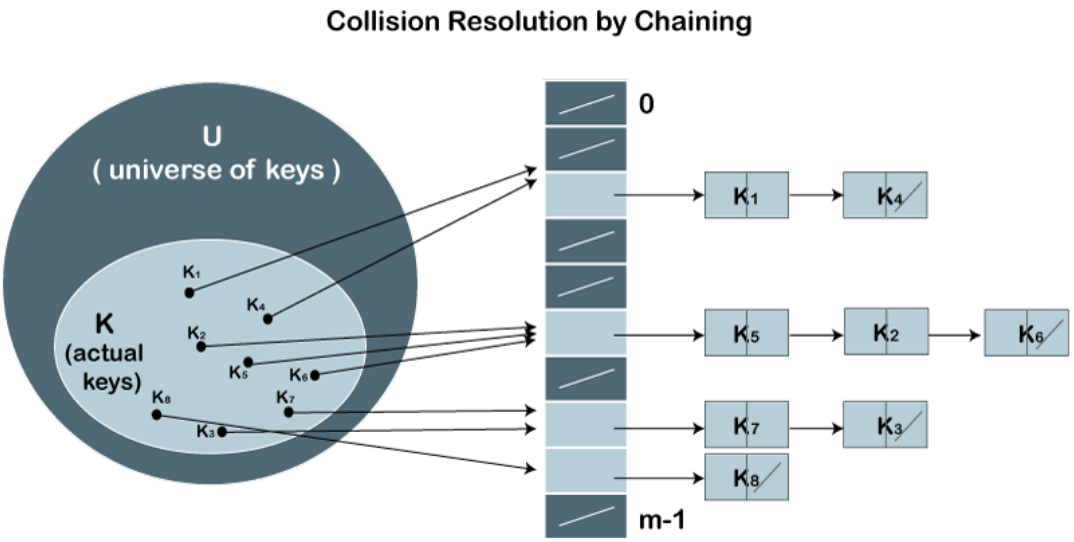
The following are ways to handle collisions -

- **Chained Hashing**
- **Open Address Hashing**

Chained Hashing

In Chained Hashing, each cell of hash table is made to point to a linked list of records that have same hash value. The hash value of the hash table act as the head nodes of the linked list. If the head node is empty, then key is added at that head node otherwise, if some key(data) is already present at that head, then new key is appended or added after that head node.

Chaining is simple, but requires additional memory outside the table.



Example: Consider a list of key values

A = 3, 2, 9, 6, 11, 13, 7, 12 where m = 10, and h(k) = 2k+3

Step 1 : The index (hash value) of key 3 is:

index = h(3) = (2(3)+3)%10 = 9. The key 3 is stored at the index 9.

Step 2: The index of key value 2 is:

index = h(2) = (2(2)+3)%10 = 7 The key 2 is stored at the index 7.

Step 3: The index of key 9 is:

index = h(9) = (2(9)+3)%10 = 1 The key 9 is stored at the index 1.

Step 4: The index of key 6 is:

index = h(6) = (2(6)+3)%10 = 5 The key 6 is stored at the index 5.

Step 5: The index of key 11 is:

index = h(11) = (2(11)+3)%10 = 5

Now, the two key values (6, 11) stored at the same index, i.e., 5. This leads to the collision problem, the chained hashing method is used to avoid the collision. Create a list and add the value 11 to this list. After the creation of the new list, the newly created list will be linked to the list having value 6.

Step 6: The index of key value 13 is:

index = h(13) = (2(13)+3)%10 = 9

Now, the two key values (3, 13) are stored at the same index, i.e., 9. This leads to the collision problem, so the chained hashing method is used to avoid the collision. Create a list and add the value 13 to this list. After the creation of the new list, the newly created list will be linked to the list having value 3.

Step 7: The index of key value 7 is:

index = h(7) = (2(7)+3)%10 = 7

Now, the two key values (2, 7) are stored at the same index, i.e., 7. This leads to the collision problem, so the chained hashing method is used to avoid the collision. Create

a list and add the value 7 to this list. After the creation of the new list, the newly created list will be linked to the list having value 2.

Step 8: The index of key value 12 is:

index = h(12) = (2(12)+3)%10 = 7

The value 12 must be stored at index 7, but the value 2 already exists at index 7. So, create a new list and add 12 to the list. The newly created list will be linked to the list having a value 7.

The calculated index value(hash value) for each key value is shown in the below table:

key	Location(u)
3	((2*3)+3)%10 = 9
2	((2*2)+3)%10 = 7
9	((2*9)+3)%10 = 1
6	((2*6)+3)%10 = 5
11	((2*11)+3)%10 = 5
13	((2*13)+3)%10 = 9
7	((2*7)+3)%10 = 7
12	((2*12)+3)%10 = 7

Open Address Hashing

The **difference between chained hashing and open address hashing** is that any amount of data can be added in chained hashing technique but in open addressing technique the data added is equal to the number of hash values present in the hash table.

Three techniques are used to resolve the collision:

1. **Linear probing**
2. **Quadratic probing**
3. **Double Hashing technique**

Linear Probing

In Linear probing, each cell in the hash table contains a key-value pair, so when the collision occurs by mapping a new key to the cell already occupied by another key, then linear probing technique searches for the closest free locations and adds a new key to that empty cell. In linear probing, searching is performed sequentially.

Example for the linear probing:

Consider a list of keys, **A = 3, 2, 9, 6, 11, 13, 7, 12 where m = 10, and h(k) = 2k+3**

Step 1: The key values 3, 2, 9, 6 are stored at the indexes 9, 7, 1, 5 respectively.

Key = 3	index = (2(3)+3)%10 = 9	The key 3 is stored at the index 9.
Key = 2	index = (2(2)+3)%10 = 7	The key 2 is stored at the index 7
Key = 9	index = (2(9)+3)%10 = 1	The key 3 is stored at the index 1
Key = 6	index = (2(6)+3)%10 = 5	The key 3 is stored at the index 5

Step 2: Key is 11. The index value = (2(11)+3)%10 = 5 cell is already occupied by another key value, i.e., 6. When linear probing is applied, the nearest empty cell to the index 5 is 6. Therefore, the value 11 will be added at the index 6.

Step 3: Key is 13. The index value = (2(13)+3)%10 = 9
cell is already filled at index 9. When linear probing is applied, the nearest empty cell to the index 9 is 0. Therefore, the value 13 will be added at the index 0.

Step 4: Key is 7. The index value = (2(7)+3)%10 = 7
cell is already filled at index 7. When linear probing is applied, the nearest empty cell to the index 7 is 8. Therefore, the value 7 will be added at the index 8.

Step 5: Key is 12. The index value = (2(12)+3)%10 = 7
The cell is already filled at index 7. When linear probing is applied, the nearest empty cell to the index 7 is 2. Therefore, the value 12 will be added at the index 2.

Step 1

Index	Key
0	
1	**9**
2	
3	
4	
5	**6**
6	
7	**2**
8	
9	**3**

Step 2

Index	Key
0	
1	**9**
2	
3	
4	
5	**6**
6	**11**
7	**2**
8	
9	**3**

Step 3

Index	Key
0	**13**
1	**9**
2	
3	
4	
5	**6**
6	**11**
7	**2**
8	
9	**3**

Step 4

Index	Key
0	**13**
1	**9**
2	
3	
4	
5	**6**
6	
7	**2**
8	**7**
9	**3**

Step 5

Index	Key
0	**13**
1	**9**
2	**12**
3	
4	
5	**6**
6	
7	**2**
8	**7**
9	**3**

Quadratic Probing

In case of linear probing, searching is performed linearly. In quadratic probing is an open addressing technique that uses quadratic polynomial for searching until an empty slot is found.

Quadratic probing allows the insertion key at first free location from **$(u+i^2)\%m$ where i = 0 to m-1.**

Example for quadratic probing

Consider list of keys

A = 3, 2, 9, 6, 11, 13, 7, 12 where m = 10, and h(k) = 2k+3

Step 1:

Key = 3	index = (2(3)+3)%10 = 9	The key 3 is stored at the index 9.
Key = 2	index = (2(2)+3)%10 = 7	The key 2 is stored at the index 7
Key = 9	index = (2(9)+3)%10 = 1	The key 3 is stored at the index 1
Key = 6	index = (2(6)+3)%10 = 5	The key 3 is stored at the index 5

The key values 3, 2, 9, 6 are stored at the indexes 9, 7, 1, 5, respectively. No need to apply the quadratic probing technique on these key values as there is no occurrence of the collision.

Step 2: Key is 11. The index value = (2(11)+3)%10 = 5

The index value of 11 is 5, but this location is already occupied by the 6. So, apply the quadratic probing technique.

When i = 0, Index= $(5+0^2)\%10 = 5$

When i=1, Index = $(5+1^2)\%10 = 6$

Since location 6 is empty, so the value 11 will be added at the index 6.

Step 3: Key is 13. The index value = (2(13)+3)%10 = 9

The index value of 13 is 9. At index 9, the cell is occupied by another value, i.e., 3. So, apply the quadratic probing technique to calculate the free location.

When i=0, Index = $(9+0^2)\%10 = 9$

When i=1, Index = $(9+1^2)\%10 = 0$

Since location 0 is empty, so the value 13 will be added at the index 0.

Step 4: Key is 7. The index value = (2(7)+3)%10 = 7

The index value of 7 is 7. At index 7, the cell is occupied by another value, i.e., 7. So, apply the quadratic probing technique to calculate the free location.

When i=0, Index = $(7+0^2)\%10 = 7$

When i=1, Index = $(7+1^2)\%10 = 8$

Since location 8 is empty, so the value 7 will be added at the index 8.

Step 5: Key is 12. The index value = (2(12)+3)%10 = 7

The index value of 12 is 7. The cell at index 7 is already occupied by the value 2. So, apply the Quadratic probing technique on 12 to determine the free location.

When i=0, Index= $(7+0^2)\%10 = 7$

When i=1, Index = $(7+1^2)\%10 = 8$

When i=2, Index = $(7+2^2)\%10 = 1$

When i=3, Index = $(7+3^2)\%10 = 6$

When i=4, Index = $(7+4^2)\%10 = 3$

Since the location 3 is empty, so the value 12 would be stored at the index 3.

The final hash table is:

0	13
1	9
2	
3	12
4	
5	6
6	11
7	2
8	7
9	3

Therefore, the order of the elements is 13, 9, _, 12, _, 6, 11, 2, 7, 3.

Double Hashing

Double hashing is an open addressing technique which is used to avoid the collisions. When the collision occurs then this technique uses the secondary hash of the key. It uses one hash value as an index to move forward until the empty location is found.

In double hashing, two hash functions are used. Suppose $h_1(k)$ is one of the hash functions used to calculate the locations whereas $h_2(k)$ is another hash function. It can be defined as "insert k_i at first free place from ***(u+v*i)%m*** where i=(0 to m-1)". In this

case, u is the location computed using the hash function ($h_1(k)$ % m) and v is equal to ($h_2(k)$%m).

Example for double probing.

A = 3, 2, 9, 6, 11, 13, 7, 12 where m = 10, and

$h_1(k)$ = 2k+3

$h_2(k)$ = 3k+1

key	Location (u)	v	probe
3	**((2*3)+3)%10 = 9**	-	1
2	**((2*2)+3)%10 = 7**	-	1
9	**((2*9)+3)%10 = 1**	-	1
6	**((2*6)+3)%10 = 5**	-	1
11	**((2*11)+3)%10 = 5**	(3(11)+1)%10 =4	3
13	**((2*13)+3)%10 = 9**	(3(13)+1)%10 = 0	
7	**((2*7)+3)%10 = 7**	(3(7)+1)%10 = 2	
12	**((2*12)+3)%10 = 7**	(3(12)+1)%10 = 7	2

Step 1: No collision would occur while inserting the keys (3, 2, 9, 6), so double hashing is not applied on these key values.

Step 2: On inserting the key 11 in a hash table, collision will occur because the calculated index value of 11 is 5 which is already occupied by some another value. Therefore, apply the double hashing technique on key 11. When the key value is 11, the value of v is 4.

Now, substituting the values of u and v in (u+v*i)%m
When i=0, Index = (5+4*0)%10 =5

When i=1, Index = (5+4*1)%10 = 9

When i=2, Index = (5+4*2)%10 = 3

Since the location 3 is empty in a hash table, therefore, the key 11 is added at the index 3.

Step 3: The next element is 13. The calculated index value of 13 is 9 which is already occupied by some another key value. So, double hashing technique is applied to find the free location. The value of v is 0.

Now, substituting the values of u and v in (u+v*i)%m

When i=0, Index = (9+0*0)%10 = 9

The value 9 is got in all the iterations from 0 to m-1 as the value of v is zero. Therefore, 13 cannot be inserted into a hash table.

Step 4: The next element is 7. The calculated index value of 7 is 7 which is already occupied by some another key value. So, double hashing technique is applied to find the free location. The value of v is 2.

Now, substituting the values of u and v in (u+v*i)%m

When i=0, Index = (7 + 2*0)%10 = 7

When i=1, Index = (7+2*1)%10 = 9

When i=2, Index = (7+2*2)%10 = 1

When i=3, Index = (7+2*3)%10 = 3

When i=4, Index = (7+2*4)%10 = 5

When i=5, Index = (7+2*5)%10 = 7

When i=6, Index = (7+2*6)%10 = 9

When i=7, Index = (7+2*7)%10 = 1

When i=8, Index = (7+2*8)%10 = 3

When i=9, Index = (7+2*9)%10 = 5

After checking all the cases of i (from 0 to 9), suitable place to insert 7 is not found. Therefore, key 7 cannot be inserted in a hash table.

Step 5: The next element is 12. The calculated index value of 12 is 7 which is already occupied by some another key value. So, double hashing technique is used to find the free location. The value of v is 7.

Now, substituting the values of u and v in (u+v*i)%m

When i=0, Index = (7+7*0)%10 = 7

When i=1, Index = (7+7*1)%10 = 4

Since the location 4 is empty, therefore, the key 12 is inserted at the index 4.

The final hash table would be:

0	
1	9
2	
3	11
4	12
5	6
6	
7	2
8	
9	3

The order of the elements is _, 9, _, 11, 12, 6, _, 2, _, 3.

Basic Operations

Following are the basic primary operations of a hash table.

- **Search** − Searches an element in a hash table.
- **Insert** − inserts an element in a hash table.
- **delete** − Deletes an element from a hash table.

Application of hashing

- Message digest
- Password verification
- Compiler operation
- Data Structures (Programming language)

~~~~~~
~~~~~~

Binary Tree	Binary Search Tree
A Binary Tree is a non-linear data structure in which a node can have 0, 1 or 2 nodes. Each node consists of a left pointer, right pointer and data element.	A Binary Search Tree is an organized binary tree with a structured organization of nodes.
There is no node structure in binary tree.	The values of left subtree should be smaller than root and the right subtree values should be greater.
The operations that can be performed are deletion, insertion and traversal	As these are sorted binary trees, they are used for fast and efficient binary search, insertion and deletion.
There are several types. Most common ones are the Complete Binary Tree, Full Binary Tree, Extended Binary Tree	The most popular ones are AVL Trees, B-Tree, T-Trees.

Stack	Queue
Stack works on concept Last In First Out	Queue works on concept First In First Out
Data is inserted and deleted from **top** end.	Data is inserted from **rear** end and removed from **front** end.
Stack operations are called push and pop	Queue operations are called Enqueue and Dequeue
Stack is visualized as vertical collections	Queue is visualized as horizontal collections
Example: Collections of plates on dinning table.	Example: people standing in ticket counter

BFS VERSUS DFS

BFS	DFS
A graph traversal algorithm that starts traversing the graph from the root node and explores all the neighboring nodes	An algorithm that starts with the initial node of the graph and then goes deeper and deeper until finding the required node or the node which has no children
Stands for Breadth First Search	Stands for Depth First Search
Uses queue	Uses stack
Consumes more memory	Consumes les memory
Focuses on visiting the oldest unvisited vertices first	Focuses on visiting the vertices along the edge in the beginning
Applications of BFS: i) To find shortest path. ii) In Spanning tree. iii) In Connectivity.	Applications of DFS i)Useful in Cycle detection. ii)In Connectivity testing. iii)In finding Spanning trees & forest.
5. The Time complexity of BFS is O(V + E), where V stands for vertices and E stands for edges.	The Time complexity of DFS is also O(V + E), where V stands for vertices and E stands for edges.